Project 2 — Hubble Ultra Deep Field: Photometric & Spectroscopic Redshift Comparison

Course: Astro 142 — Data & Computation in Astronomy
Author: *Isaiah Mills-Terry*
Date: Fall 2025

---

1. Overview

This project performs data retrieval, catalog cross-matching, RGB mosaic imaging, and visualization of galaxy redshifts in the **Hubble Ultra Deep Field (HUDF)**. Using data from **MAST** (NASA Mikulski Archive for Space Telescopes) and catalogs hosted at **VizieR**, I produced:

- A **3-color RGB mosaic** of the HUDF using *HST/ACS/WFC* imaging.
- Overlaid **photometric** and **spectroscopic** galaxy redshift positions.
- A **joint distribution plot** comparing photometric vs. spectroscopic redshifts.
- A **multi-panel inset mosaic** showing zoom-in regions of the HUDF (developed on a Git branch and later merged).
- A codebase that includes **logging**, **exception handling**, **sanitized inputs**, and clear **documentation strings**, as required in the project rubric
  proj2-251023 (3)
  .

---

2. Imaging Data: HST ACS/WFC 3-Color Mosaic

Instrument

- **Hubble Space Telescope (HST)**

- **Advanced Camera for Surveys — Wide Field Channel (ACS/WFC)**

Filters Used For RGB Composite

Color Channel	Filter	Approx. Wavelength
Blue	F435W	~0.43 μm (B-band)
Green	F606W	~0.59 μm (Broad V)
Red	F775W	~0.77 μm (i-band)

These bandpasses were chosen because they provide the closest mapping to visual RGB color perception for optical HST imaging.

Mosaic Generation Steps

1. Queried MAST for HUDF observations using:
    - `obs_collection="HST"`
    - `instrument_name="ACS/WFC"`
    - `filters = ['F435W', 'F606W', 'F775W']`
    - `dataproduct_type="image"`
2. Filtered the product list to include:
    - `productType="SCIENCE"`
    - `Drizzled mosaics (DRZ files).`
3. Downloaded one mosaic per filter.
4. Performed percentile clipping & normalization for each filter before stacking into an RGB array.
5. Displayed the mosaic using WCS projection to ensure correct RA/Dec coordinates.

Reference:

HST dataset accessed via MAST: https://mast.stsci.edu

---

3. Catalog Retrieval (Photometric and Spectroscopic Redshifts)

Two catalogs were downloaded from **VizieR** using `astroquery.vizier:`

Photometric Redshift Catalog

- **`VizieR Catalog:`** `J/AJ/150/31`
- Contains photo-z values (z_{phot}) derived from broad-band SED fitting.

Spectroscopic Redshift Catalog

- **`VizieR Catalog:`** `J/A+A/608/A2`
- Part of the **MUSE HUDF DR2** spectroscopic survey.
- Contains high-confidence redshifts (z_{spec}) from emission/absorption line fitting.

Both catalogs are stored locally as FITS files:

```
Photoz_catalog.fits
Spec_catalog.fits
```

---

4. Coordinate Matching

To identify galaxies with **both** photometric and spectroscopic redshift measurements, I used:

- **`SkyCoord objects`** `for RA/Dec conversion`
- **`match_to_catalog_sky`** `for nearest-neighbor sky matching`
- A matching tolerance of **0.5 arcseconds**

This produced a table containing matched entries with:

- Spec coordinates
- Photometric coordinates
- Angular separation
- z□□$_{ec}$ and z□□$_{o}$□ values

This enabled plotting the **joint redshift distribution** (z□□$_{ec}$ vs. z□□$_{o}$□).

5. Main Mosaic Plot With Galaxy Overlays

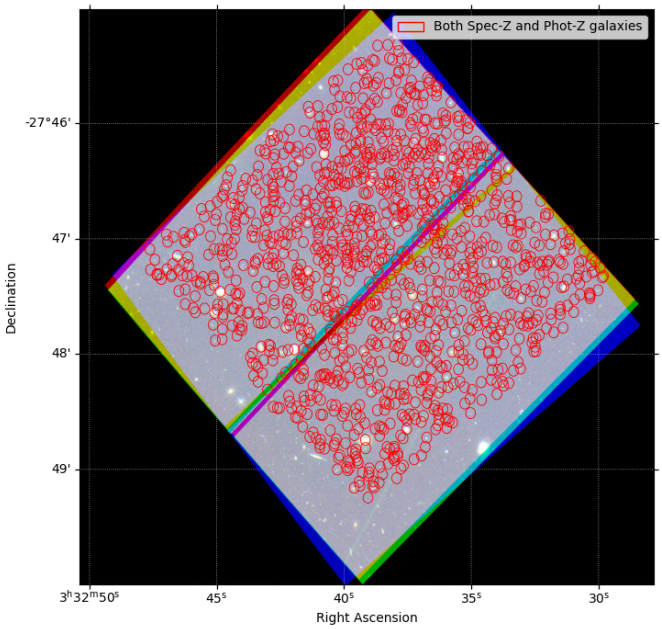
On top of the RGB mosaic, I overplotted galaxies:

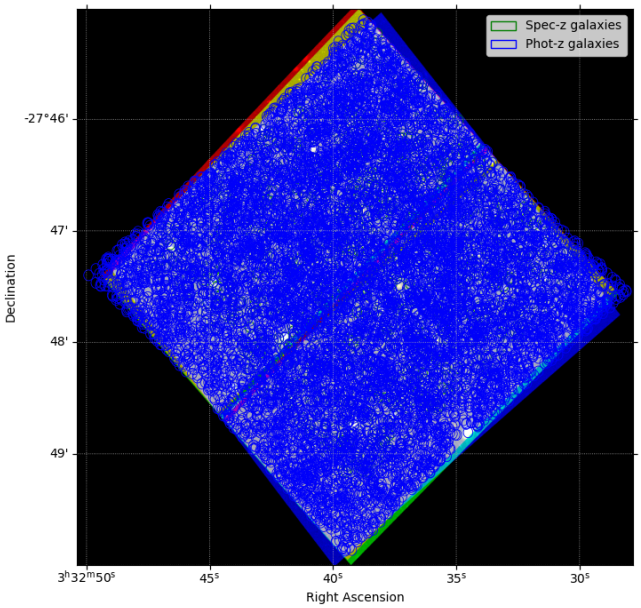
Marker	Meaning
Blue circles	Photometric-only redshift sources
Green circles	Spectroscopic redshift sources
Both	Photometric + spectroscopic matches

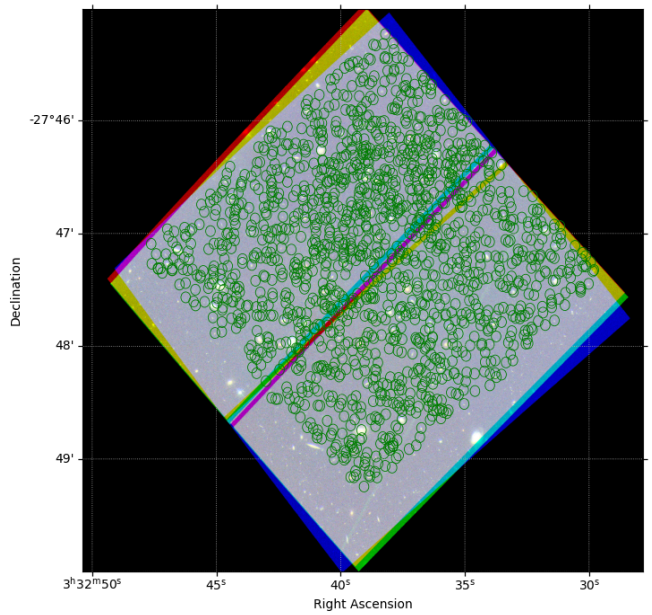
To place them correctly, each circle uses:

```
transform = ax.get_transform("world")
```

This ensures overlays use the **WCS coordinate grid**, not pixel coordinates.


Both Spec-Z and Phot-Z galaxies
-27°46'
47'
48'
49'
Declination
3h32m50s
45s
40s
35s
30s
Right Ascension


Spec-z galaxies
Phot-z galaxies
-27°46'
47'
48'
49'
Declination
3h32m50s
45s
40s
35s
30s
Right Ascension



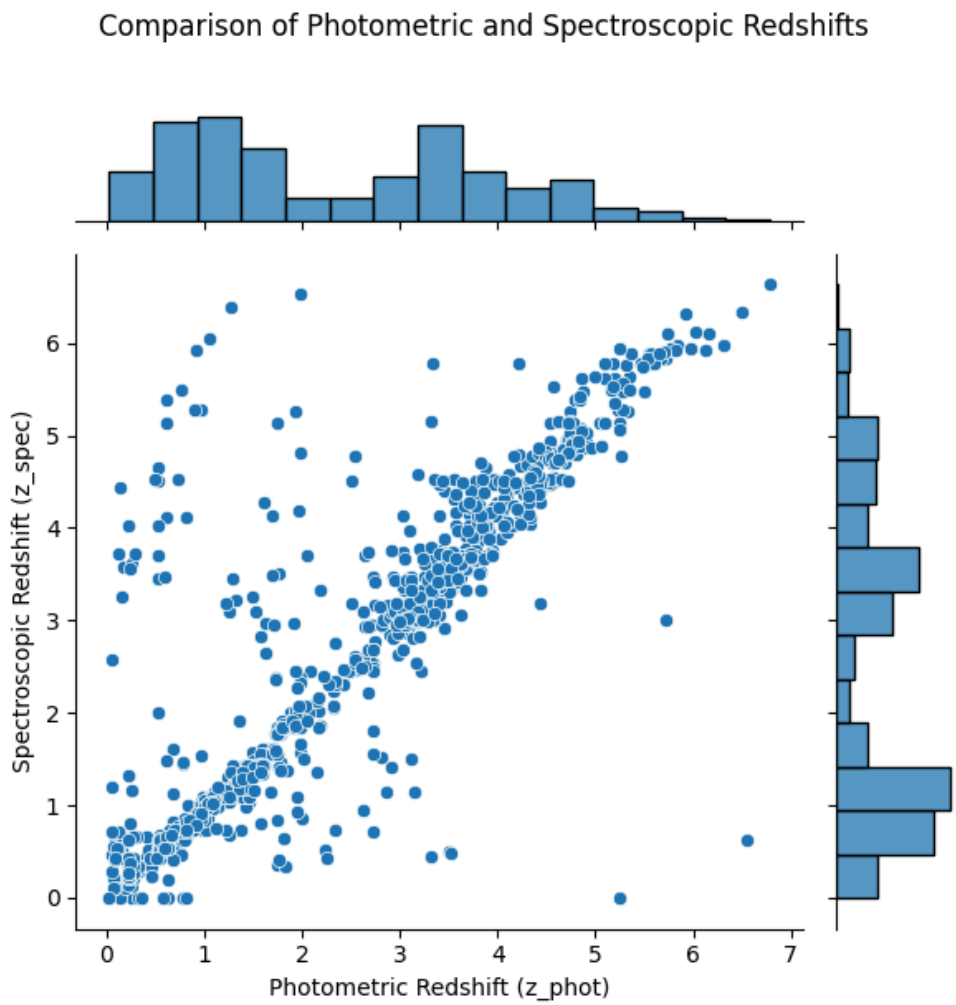
6. Joint Distribution Plot

For matched photo-z and spec-z sources, I created a scatter plot of:

```
z_spec vs. z_photo
```

with a 1:1 reference line to visualize deviations.

This allows quick identification of outliers and regions where photomet



ric redshifts struggle.

. Multi-Panel Inset Layout (Git Branch Requirement)

Per project instructions, I created a development branch:

```
git branch inset-layout
git switch inset-layout
```

Then I:

1. Wrote a helper function `add_inset(ax, region)` that:

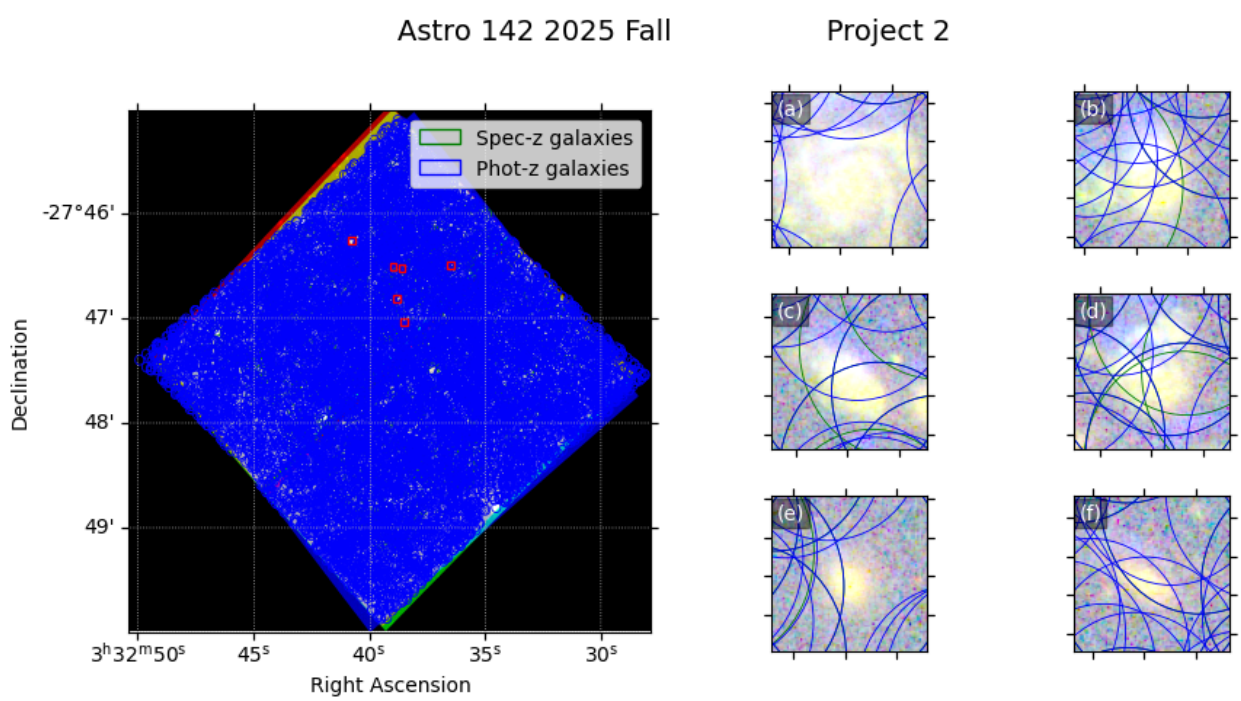
- Cuts out a WCS-defined subregion
- Plots it in a new panel
- Draws the bounding box on the main mosaic
- Labels each panel `(a), (b), (c)`

2. Added three example inset panels, each zooming into a dense galaxy region.

After finalizing the inset panels:

```
git add .
git commit -m "Added inset-panel mosaic layout"
git switch main
git merge inset-layout
```

This successfully merged the feature branch into `main`.



8. Error Handling & Logging

Logging

Throughout both scripts I used:

```
logging.info()
logging.warning()
logging.error()
logging.exception()
```

Examples include:

- Logging the number of MAST observations returned.
- Logging catalog retrieval status.
- Logging successful and failed filter downloads.

Exception Handling

I wrapped all major operations in `try/except` blocks, including:

- MAST queries
- DRZ mosaic filtering
- Catalog retrieval
- FITS file writing
- Product downloading

Failing one filter does not crash the program — it logs the error and continues.

Input Sanitization

Functions check:

- That tables are non-empty

- That expected catalog keys exist
- That WCS headers contain necessary components

This satisfies the rubric requirements for robust code behavior.

---

9. References

Data Sources

- MAST: HST ACS/WFC imaging (F435W, F606W, F775W)
- VizieR Photometric Catalog: `J/AJ/150/31`
- VizieR MUSE Spectroscopic DR2 Catalog: `J/A+A/608/A2`

Literature / ADS Exploration

- Bacon et al. 2017, *MUSE HUDF Survey*
- Beckwith et al. 2006, *Hubble Ultra Deep Field*

10. Code Appendix

Project 2 - RGB Field Query.py > ...

```python

import numpy as np
from astropy.io import fits
from astropy.wcs import WCS
import matplotlib.pyplot as plt
from astropy import units as u
from astropy import visualization
import astropy.constants as const
from astropy.table import Table, QTable
from astropy.time import Time, TimeDelta
from astropy.coordinates import SkyCoord, EarthLocation, AltAz, FK5, Galactic
from astropy.visualization import quantity_support, time_support
import matplotlib.pyplot as plt
import numpy as np
from astroquery.mast import Observations
from pprint import pprint
from astropy.table import Table
import pandas as pd
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s"
)


# Get Center of Hubble UDF
hudf = SkyCoord("03h32m39.0s -27d47m29.0s", frame="icrs")

# ---------------- Query MAST for HST/ACS/WFC images ----------------
try:
    logging.info("Querying MAST for HST/ACS/WFC images around HUDF...")
    obs = Observations.query_criteria(
        coordinates=hudf,
        radius=3 * u.arcmin,
        obs_collection="HST",
        instrument_name="ACS/WFC",
        dataproduct_type="image",
        filters=["F606W", "F775W", "F435W"],
    )
    logging.info("MAST query returned %d observations.", len(obs))

    if len(obs) == 0:
        logging.error("No observations found for the given HUDF query.")
```

```python

except Exception:
    logging.exception("Failed during MAST query_criteria call.")
    raise

# ----------------- Get product list and filter to science mosaics ----------------
try:
    logging.info("Retrieving product list for selected observations...")
    products = Observations.get_product_list(obs)
    logging.info("Product list contains %d entries.", len(products))

    logging.info("Filtering products to SCIENCE images...")
    science_images = Observations.filter_products(
        products,
        productType="SCIENCE",
        mrp_only=False,
    )
    logging.info("After SCIENCE filter: %d products.", len(science_images))

    if len(science_images) == 0:
        logging.error("No SCIENCE products found after filtering.")
        raise RuntimeError("No SCIENCE products found for HUDF query.")

    # Filter for DRZ (drizzled) mosaics
    psg = science_images["productSubGroupDescription"]
    arr = np.array(psg.filled("")).astype(str)
    is_drz = np.char.find(np.char.upper(arr), "DRZ") != -1
    science_images = science_images[is_drz]
    logging.info("After DRZ filter: %d products.", len(science_images))

    if len(science_images) == 0:
        logging.error("No DRZ science mosaics found in product list.")
        raise RuntimeError("No DRZ science mosaics available for HUDF query.")

    # Put in pandas DF to select one product per parent_obsid
    image_df = science_images.to_pandas()
    image_df = image_df.sort_values("productFilename")
    image_df = image_df.groupby("parent_obsid", as_index=False).tail(1)

    logging.info(
        "Selected %d unique parent_obsid entries for image download.",
        len(image_df),
    )
```

```python
    image_table = Table.from_pandas(image_df)
    image_table = image_table[image_table["obs_collection"] == "HST"]
    logging.info(
        "Final image_table has %d rows after enforcing obs_collection == HST.",
        len(image_table),
    )

    if len(image_table) == 0:
        logging.error("image_table is empty after HST filter.")
        raise RuntimeError("No HST images left in image_table for download.")

except Exception:
    logging.exception("Failed while filtering and organizing MAST products.")
    raise

# ---------------- Download one FITS mosaic per filter ----------------
filters = ["F606W", "F775W", "F435W"]

for flt in filters:
    try:
        logging.info("Selecting products for filter %s...", flt)
        cur_filter_table = image_table[image_table["filters"] == flt]

        if len(cur_filter_table) == 0:
            logging.warning("No entries found in image_table for filter %s; skipping.", flt)
            continue

        # Download the first matching product for this filter
        logging.info("Downloading FITS data for filter %s...", flt)
        _dl = Observations.download_products(
            cur_filter_table[0],
            extension="fits",
            download_dir=".",
        )
        logging.info("Successfully downloaded data for filter %s.", flt)

    except Exception:
        logging.exception("Failed to download data for filter %s.", flt)
        # continue to next filter so a single failure doesn't kill the whole script
        continue
```

```python
import numpy as np
from astropy.io import fits
from astropy.wcs import WCS
import matplotlib.pyplot as plt
from astropy import units as u
from astropy import visualization
import astropy.constants as const
from astropy.table import Table, QTable
from astropy.time import Time, TimeDelta
from astropy.coordinates import SkyCoord, EarthLocation, AltAz, FK5, Galactic
from astropy.visualization import quantity_support, time_support
import matplotlib.pyplot as plt
import numpy as np
from astroquery.mast import Observations, Catalogs
from pprint import pprint
from astropy.table import Table
import pandas as pd
from astroquery.simbad import Simbad
from astroquery.vizier import Vizier
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s"
)

vizier = Vizier(row_limit=-1)

# ---------------- Catalog lookup with logging + error handling ----------------

spec_catalog_key = "J/A+A/608/A2"
phot_catalog_key = "J/AJ/150/31"

try:
    logging.info("Searching Vizier for MUSE HUDF DR2 catalogs...")
    catalog_list = vizier.find_catalogs("The MUSE Hubble Ultra Deep Field surveys: Data release II")
    logging.info("Found %d matching catalogs.", len(catalog_list))

    # Optional: keep this print so you can see what was found
    for k, v in catalog_list.items():
        print(k, ":", v.description)
```

```python
    # Sanity check: make sure our expected keys are actually present
    if spec_catalog_key not in catalog_list:
        logging.error("Spectroscopic catalog key %s not found in Vizier results.", spec_catalog_key)
        raise KeyError(f"Missing expected spectroscopic catalog: {spec_catalog_key}")

    if phot_catalog_key not in catalog_list:
        logging.error("Photometric catalog key %s not found in Vizier results.", phot_catalog_key)
        raise KeyError(f"Missing expected photometric catalog: {phot_catalog_key}")

except Exception as e:
    logging.exception("Failed during Vizier catalog search.")
    raise  # re-raise so the notebook / script stops in a controlled way

# ----------------- Retrieve the specific catalogs -----------------

try:
    logging.info("Retrieving spectroscopic catalog %s from Vizier...", spec_catalog_key)
    spec_cat = vizier.get_catalogs(spec_catalog_key)
    logging.info("Retrieved %d spectroscopic tables.", len(spec_cat))

    logging.info("Retrieving photometric catalog %s from Vizier...", phot_catalog_key)
    phot_cat = vizier.get_catalogs(phot_catalog_key)
    logging.info("Retrieved %d photometric tables.", len(phot_cat))

except Exception:
    logging.exception("Failed to download one or more catalogs from Vizier.")
    raise

# Combined spectroscopic table (not MOSAIC or UDF10 subsets)
if len(spec_cat) == 0:
    logging.error("Spectroscopic catalog list is empty after download.")
    raise RuntimeError("No spectroscopic tables returned from Vizier.")

spec_cat_table = spec_cat[0]

# Inspect the photometric tables (you can still see one example row/columns)
for tbl in phot_cat:
    pprint(tbl[0])
    print(tbl[0].columns)

if len(phot_cat) == 0:
    logging.error("Photometric catalog list is empty after download.")
    raise RuntimeError("No photometric tables returned from Vizier.")
```

```python
phot_cat_table = phot_cat[0]

logging.info(
    "Final spectroscopic table has %d rows and %d columns.",
    len(spec_cat_table), len(spec_cat_table.colnames)
)
logging.info(
    "Final photometric table has %d rows and %d columns.",
    len(phot_cat_table), len(phot_cat_table.colnames)
)

# ----------------- Save the tables to disk with error handling -----------------

try:
    # Clear metadata that can confuse downstream FITS readers
    phot_cat_table.meta.clear()
    spec_cat_table.meta.clear()

    logging.info("Writing photometric catalog to Photoz_catalog.fits ...")
    phot_cat_table.write("Photoz_catalog.fits", overwrite=True)
    logging.info("Successfully wrote Photoz_catalog.fits")

    logging.info("Writing spectroscopic catalog to Spec_catalog.fits ...")
    spec_cat_table.write("Spec_catalog.fits", overwrite=True)
    logging.info("Successfully wrote Spec_catalog.fits")

except Exception:
    logging.exception("Failed while writing FITS files to disk.")
    raise
```